their code. DHH explains on Signal v. Noise how Hey decided to give back to the web by allowing their source to be viewable.[64]

What our resetOnOutsideClick method is doing is taking the event.target, which is the item being clicked, and this.element, which is the DOM event of the controller, and calling reset if the target is not in the element. That is to say, any click outside the search form or result element will close the element. The reset method hides the result element, sets its text to blank (so that it doesn't show past results when redisplayed), and sets the input target—the text field itself—to blank.

Debouncing User Input

With that feature, we can now change our submit to reset if the input is blank and close the window. We can also take care of the networking problem of results getting crossed, using a procedure often called *debouncing*.

Take a look at this code:

chapter_08/02/app/javascript/controllers/search_controller.ts

```
basicSubmit(): void {
  if (this.inputTarget.value === "") {
    this.reset()
  } else {
    this.formTarget.requestSubmit()
  }
}

submit = this.debounce(this.basicSubmit.bind(this))

debounce(functionToDebounce: Function, wait = 300) {
  let timeoutId = null

  return (...args: any[]) => {
    clearTimeout(timeoutId)
    timeoutId = setTimeout(() => {

      timeoutId = null
```